

Vario Product Specification & AI Engineering Blueprint

This document contains the complete product architecture, feature inventory, and master handoff prompt for **Vario** (the multi-variant travel planning and shared expense web application).

Part 1: Product Architecture & Feature Inventory

1. Application Overview

Vario is a desktop-first web application designed to replace multi-tab travel spreadsheets with a dynamic, multi-variant itinerary graph and a shared expense engine. Unlike traditional travel apps that enforce a rigid, linear calendar, Vario treats a trip timeline as an adjustable chain of nodes. Any single time block can host multiple alternative options (variants) that can be swapped instantly to recalculate time budgets, total costs, and weather dependencies across the entire trip day.

2. Comprehensive Feature Matrix

🗺️ Dynamic Itinerary & Multi-Variant Engine

- * **Micro-Step Granularity:** Breaks down a single transit or activity segment into precise, sequential sub-actions (e.g., Walk 9 mins -> Subway M1 30 mins -> Transfer 3 mins -> Shuttle 20 mins).
- * **Parallel Variant Toggling:** Enables users to build and store alternative paths (e.g., Variant A: Public Transit vs. Variant B: Uber/Taxi) for any given itinerary slot.
- * **Downstream Temporal Shifting:** Modifying a departure time or toggling an active variant automatically recalculates and shifts the estimated arrival and departure times of all subsequent events for that day.
- * **Hard Checkpoint Guardrails:** Allows pinning absolute time milestones (e.g., Flight Boarding at 10:00 AM, Train departure, Cruise line check-in). The app calculates a real-time buffer margin and triggers an amber or red visual alert if the currently active chain of variants breaches the safety threshold.

💵 Tri-Currency Core

- * **Global Currency State:** Instantly switches the entire user interface across three distinct contextual currency layers: Home Currency (e.g., RON), Local Destination Currency (e.g., EUR), and an International Baseline (e.g., USD).
- * **Cached Cross-Conversion:** Synchronizes an exchange rate matrix (using reliable open sources like the European Central Bank) at application initialization, executing all conversions client-side for zero latency when toggling views.

💰 Shared Expense Ledger & Debt Splitter

- * **Pre-Trip vs. Mid-Trip Classification:** Separates foundational upfront costs (flights, accommodations) from active on-the-ground spending (meals, localized public transit tickets).

- * ****Multi-Payer Attribution:**** Records exactly who paid for an expense, what currency was used at the point of sale, and assigns custom split matrices (Equally, Custom Percentages, or Fixed Shares).
- * ****Variant Cost Synchronization:**** Seamlessly injects the estimated transport or ticketing costs of the **currently active** itinerary variants into the overall projected trip financial forecast.
- * ****Balance Settlement Engine:**** Calculates running net balances for all participants and yields clear, optimized peer-to-peer settlement steps (e.g., "Andrei owes Ioana 140 EUR"), minimizing the total number of transactions required to settle the vacation.

Part 2: The Master AI Handoff Prompt

System Prompt: Project "Vario" Travel Planner Specification

You are an expert software architect and product engineer. You are tasked with helping build "Vario", a desktop-first web application designed to replace complex Excel sheets used for granular, multi-variant vacation planning and shared expense tracking.

1. Product Vision & Structural Concept

Vario allows users to plan a trip down to micro-logistical steps (e.g., transfers, walking segments) while maintaining alternative paths ("Variants") for any given slot in a day's timeline. Swapping a variant or changing a departure time dynamically recalculates downstream schedules, real-time checkpoint buffers, and total costs.

2. Core Domain Data Model

Your implementations must strictly respect this hierarchical relationship structure:

- ****TRIP:**** Global container. Houses participants, metadata, global currency preferences, and aggregated budgets.
- ****DAY:**** Holds a calendar date anchor within a Trip.
- ****ITINERARY_SLOT:**** A chronological segment within a day (e.g., "Morning Transit to Airport").
- ****VARIANT_NODE:**** Alternative configurations for a single Itinerary Slot. One variant is flagged as ACTIVE.
 - Holds: Calculated Duration, Est. Cost, Local Weather State, and an ordered array of MICRO_STEPS.
- ****MICRO_STEP:**** Granular actions inside a variant (e.g., type: 'walk', 'subway', 'car', 'flight'; attributes: duration, distance, localized cost).
- ****EXPENSE_ITEM:**** Financial entries in the ledger mapping back to a Trip, categorized by Phase (Pre-Trip, Mid-Trip), Payer ID, Currency Code, Amount, and a Split Definition Map.

3. High-Priority Functional Specifications

A. The Ripple-Effect Timeline Engine

- Users must be able to change the start time of the day's first node.
- Every subsequent node's start and end times must be calculated dynamically by summing the preceding node's duration and active variant choices.

- Hard Checkpoints (e.g., Flight Departures) must validate against the incoming active variant's calculated arrival time. If (Arrival Time is greater than Checkpoint Time minus Safety Buffer), flag a critical UI error state.

B. Tri-Currency Presentation Layer

- The system must track financial values natively in their input currency, but support an instant UI-wide toggle between three target codes: Home, Local, and International Standard.
- Exchange rates must be fetched globally at application load and applied downstream via utility conversion functions to prevent redundant database or API calculations.

C. Splitwise-Style Balance Engine

- Track who paid what, in any currency.
- Compute fractional debts among participants based on split profiles.
- Dynamically add the "Active Itinerary Variant Cost" into the overall projected total trip expenses, splitting it equally among all members automatically.
- Provide a clean settlement ledger showing the minimum number of transactions required to resolve debts.

YOUR TASK

Acknowledge your understanding of this architectural specification. Based on this system model, please fulfill the following specific implementation request:

[INSERT USER REQUEST HERE - e.g., "Design the SQL relational database schema with foreign keys" / "Generate the state management logic using React Context" / "Draft the OpenAPI 3.0 specification for the itinerary endpoints"]